



Ready, set, publish!

Write your first  python[™] Package 

PyCon 2026 - FH Burgenland - Linda Kolb



Linda

- afraid of snakes, but Python is ok 🐍
- Python Developer > 7 years
- Lead Data Scientist at SupplyBrain





Vibe Check

- Who has ever **copy-pasted Python code** from one project to another?
- And then spent time figuring out how to make it actually run (dependencies, imports)?





ETL Pipeline

ETL: extract - transform - load



extract



transform



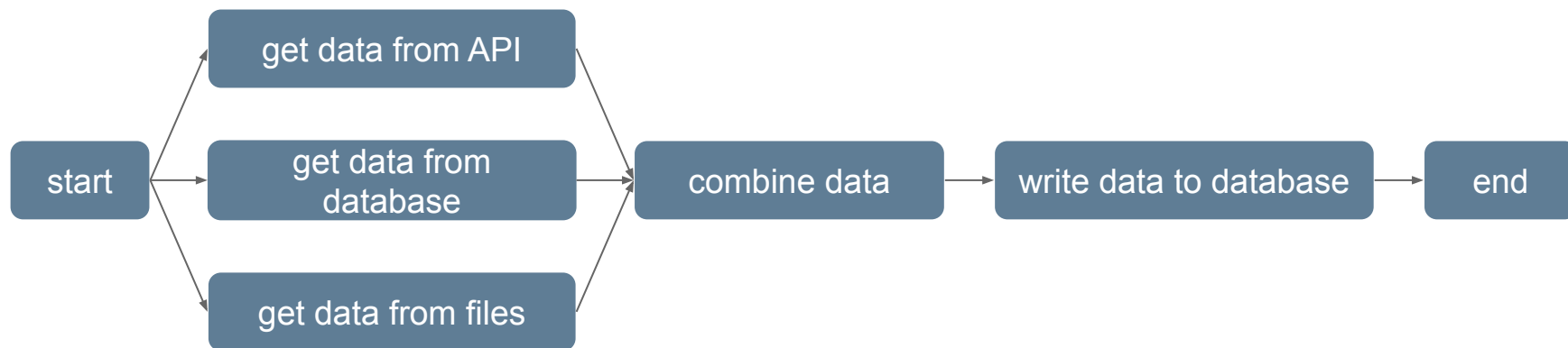
load



visualise

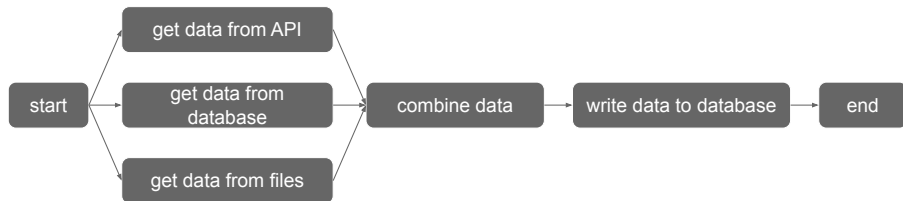
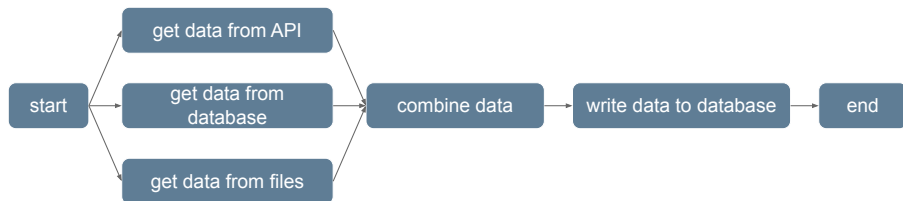
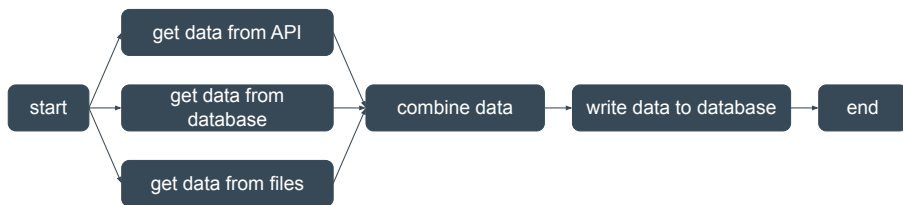


ETL Pipeline

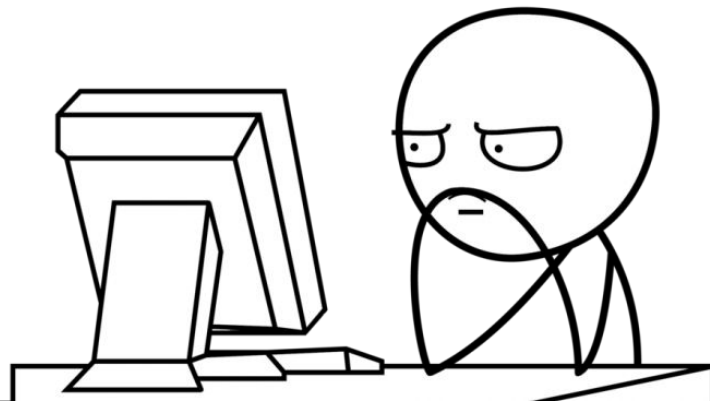




ETL Pipeline

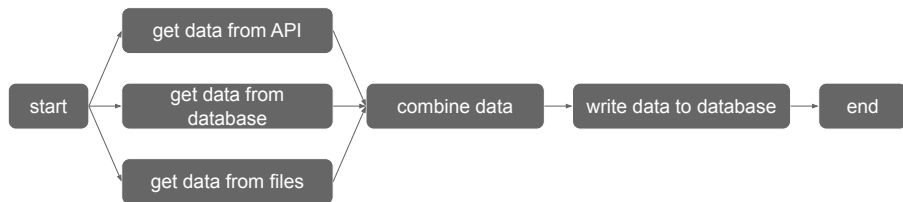
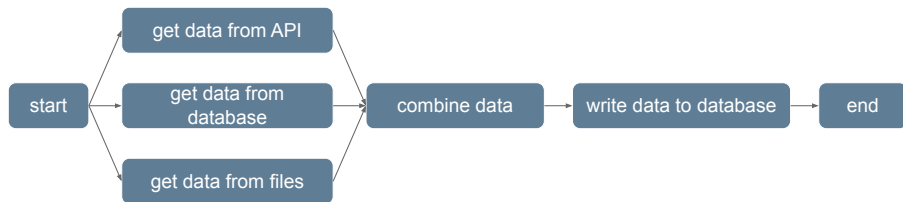
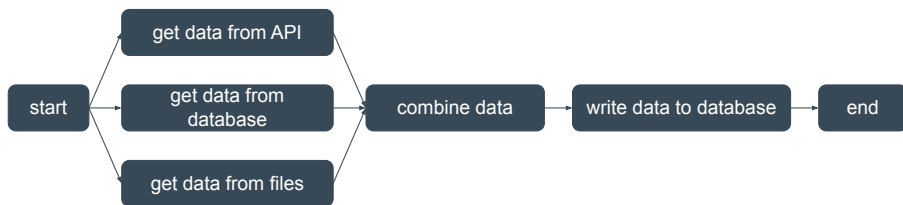


I am always
using the same
code snippets...

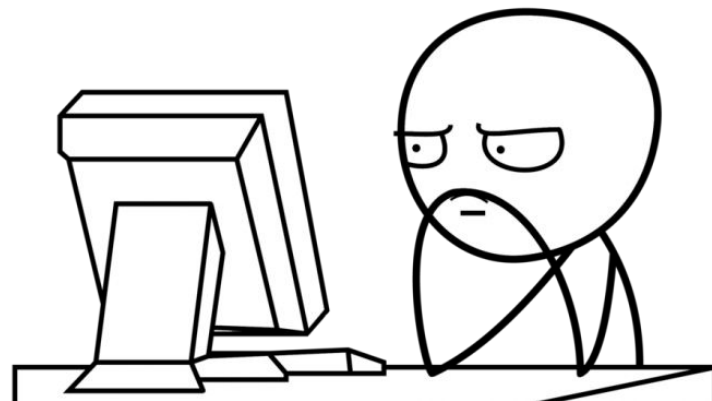




ETL Pipeline



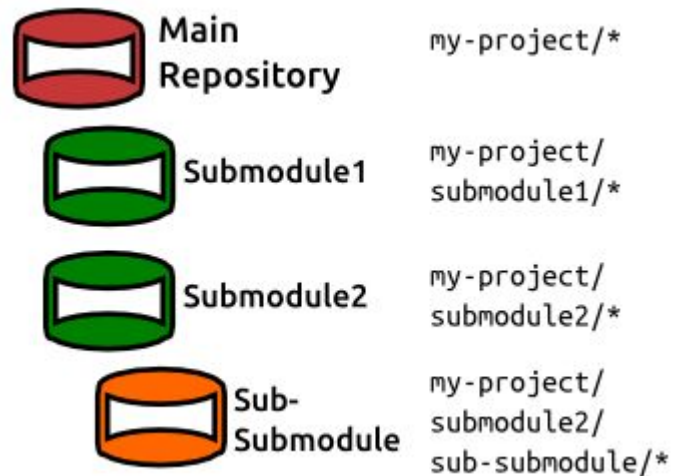
There must be some way to easily re-use this code...





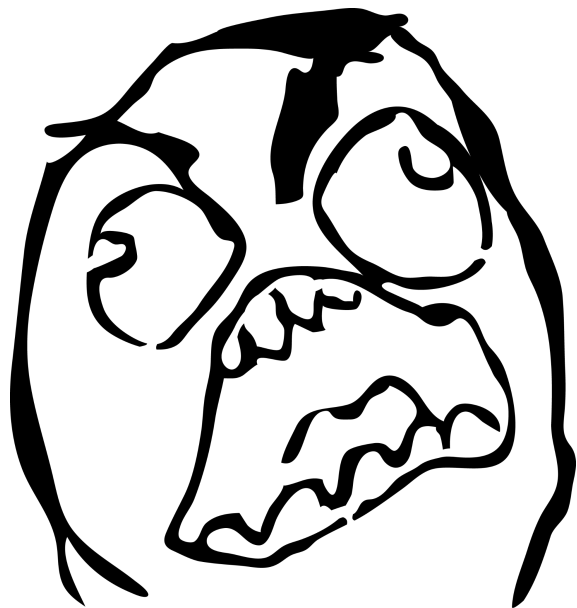
How about
git
submodules?

A Git submodule is a way to **include**
another Git repo inside your project.





But what about the
Python
dependencies??



It is possible to add a
`requirements.txt/pyproject.toml` file with
the submodule, but there is a much cleaner
way...



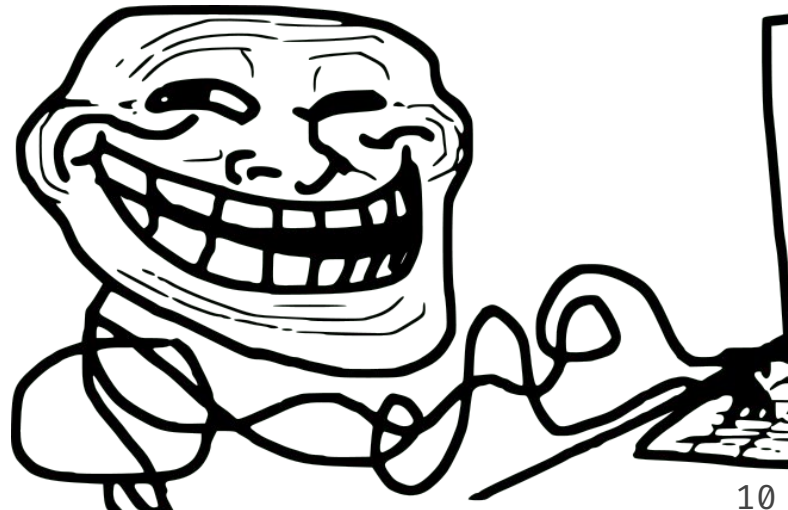


Advantages of a **Python package**:

- Designed for reuse
- Versioning
- Clean installation
- Dependency resolvment
- Easy collaboration

And the **most important thing**: You can release your own tiny product to the world!

How about writing
your own Python
package...





01 Python Package Structure

What is a Python package?

02 Python Package Dependency Management

uv vs. poetry vs. pip install

03 Testing & Pre-Commit Hooks

pytest vs. unittest and pre-commit

04 Build & Publish

Build and publish the package to a Python package registry

05 Package Integration

Install and use the package in another Python project

06 Dev Containers

Set up a Dev Container for your Python package development



Pick your Poison



Visual Studio Code



Cursor IDE

An AI-powered code editor



01

Python Package Structure

What is a Python package?



Python Packages

What is a Python Package?

- A **collection of Python code** you can reuse
- Organised in modules (files) and folders
- Can be **imported** into other projects



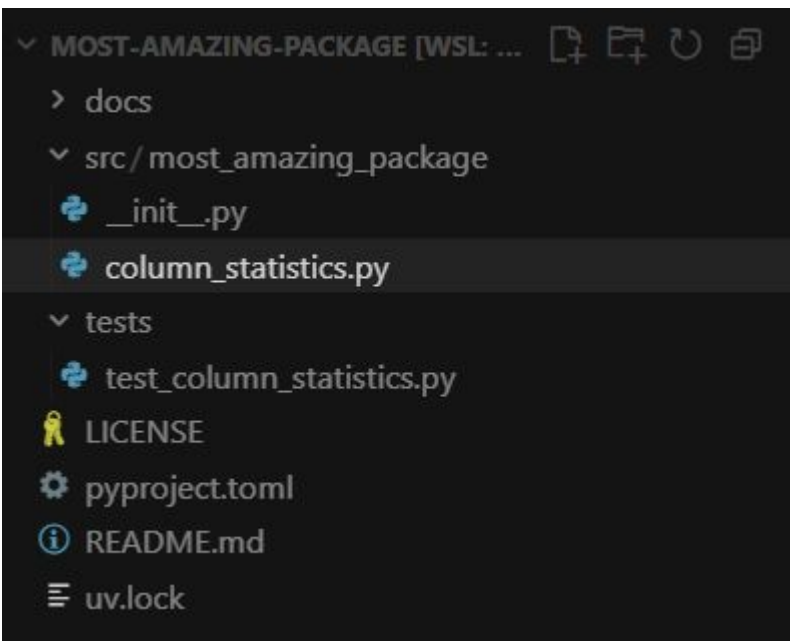
Why?

- Reuse code across projects
- Avoid copy-paste
- Share code with others
- Manage dependencies & versions



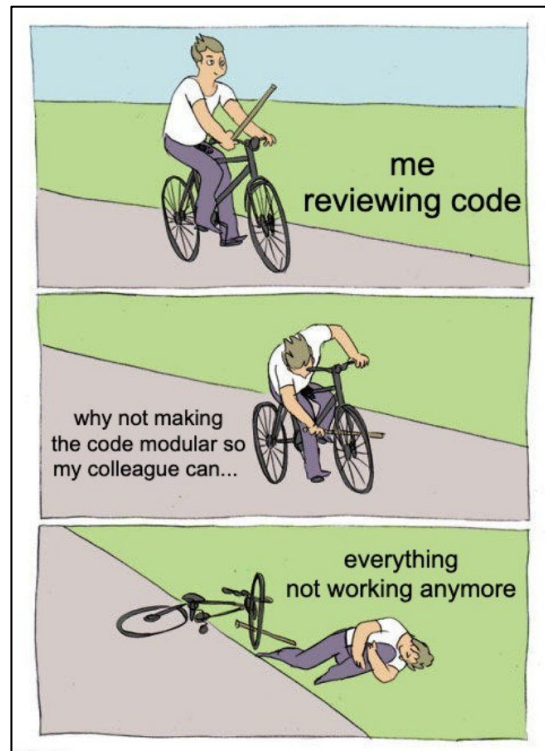
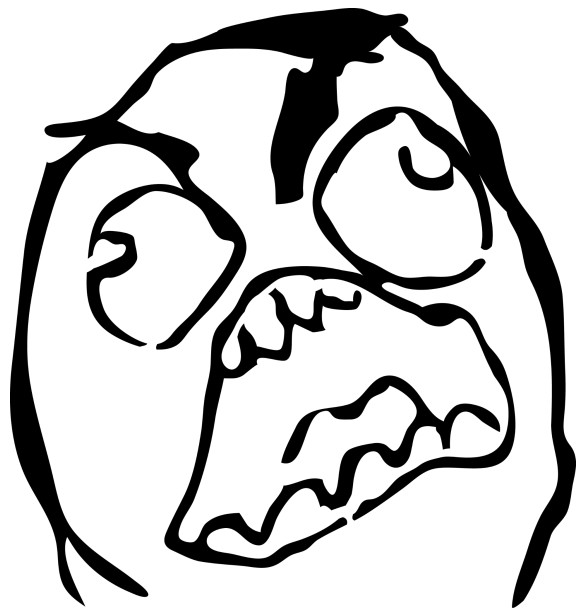


Typical Structure





But what about
Python modules??





Module vs. Package vs. Library

Module

reusable, importable
code



Package

consists of several
modules



Library

“unpackaged” package



TMP



TEMP_



TEMPORARY

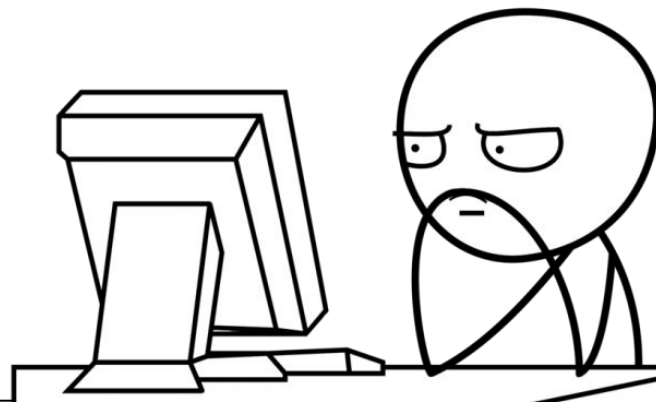


THIS VARIABLE WILL ONLY
BE USED FOR
A SHORT PERIOD OF TIME



imgflip.com

But how to
name it...

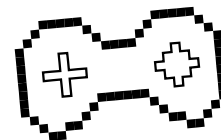




PEP 8 - Naming Convention

PEP: Python Enhancement Proposal

- Use **lowercase letters.**
- **Hyphens** are common for distribution/package names on PyPI, while Python import names typically use underscores, e.g.
`most_amazing_package`
- Use simple and short names
- Do not use abbreviations
- Make sure that the project or package names **do not conflict** with the existing Python standard library modules, e.g. `pathlib`





02

Python Package Dependency Management

`uv vs. poetry vs. pip install`



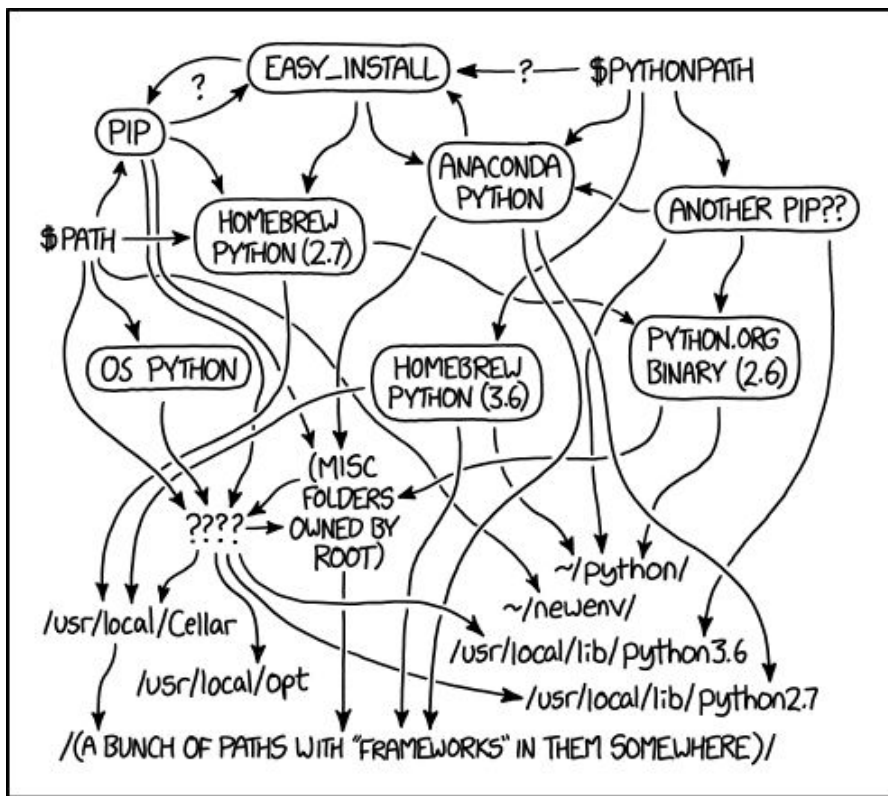
pip install vs. poetry vs. uv

	pip install	poetry	uv
Initial release	2008	2018	2024
Project configuration and lockfile	command line, requirements.txt (manual pip freeze)	pyproject.toml poetry.lock	pyproject.toml uv.lock
Virtual Environment Management	External (venv, virtualenv)	integrated	integrated
Performance	moderate	moderate	Very Fast (10-100x pip)
Build and publish	Setuptools + twine	Integrated (poetry build/publish)	Integrated (uv build/publish)



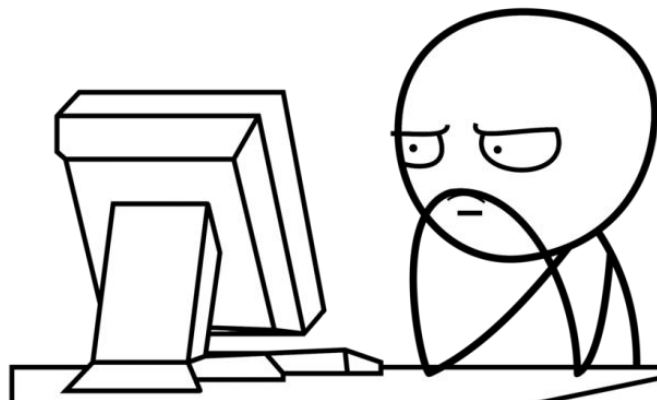
```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12
python3.12 -m venv .venv
source .venv/bin/activate
pip-compile requirements.in -o requirements.txt
pip install -r requirements.txt
```

uv sync



MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED
THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.

Pip, poetry,
uv...what about
anaconda??

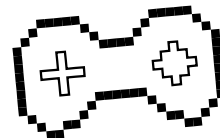




Conda (Anaconda, Miniconda)

Environment and system package manager (Python + **non-Python** dependencies)

- Packages come primarily from conda repositories
- Anaconda
 - **Proprietary** license
 - Will already include many pre-installed packages (data science focus)
- Miniconda
 - Free to use
 - Minimal installer





uv Cheat Sheet

- `uv python install 3.13`
- `uv self update 0.11.6`
- `uv init`
- `uv venv`
- `uv sync`
- `uv add requests`
- `uv add pytest -- group dev`
- `uv remove requests`
- `uv run main.py`
- `uv run pytest`
- `uv lock`
- `uv tree`
- `uv build`
- `uv publish`
- `uvx apache-airflow standalone`

Docs: <https://docs.astral.sh/uv/getting-started/features/>



Install uv 0.11.6

Linux + Mac

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Specific version:

```
curl -LsSf https://astral.sh/uv/0.11.6/install.sh  
| sh
```

Windows

```
powershell -ExecutionPolicy ByPass -c "irm  
https://astral.sh/uv/install.ps1 | iex"
```

Specific version:

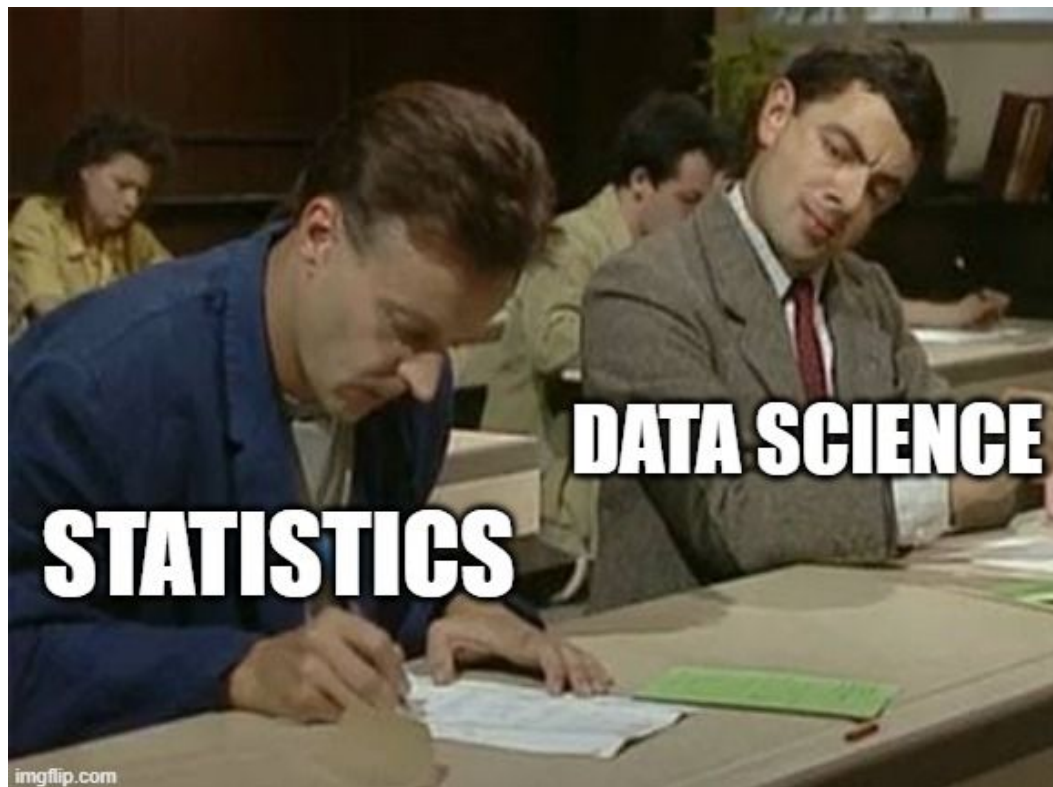
```
powershell -ExecutionPolicy ByPass -c "irm  
https://astral.sh/uv/0.11.6/install.ps1 | iex"
```

Verify with: `uv --version`

Docs: <https://docs.astral.sh/uv/getting-started/installation/#standalone-installer>



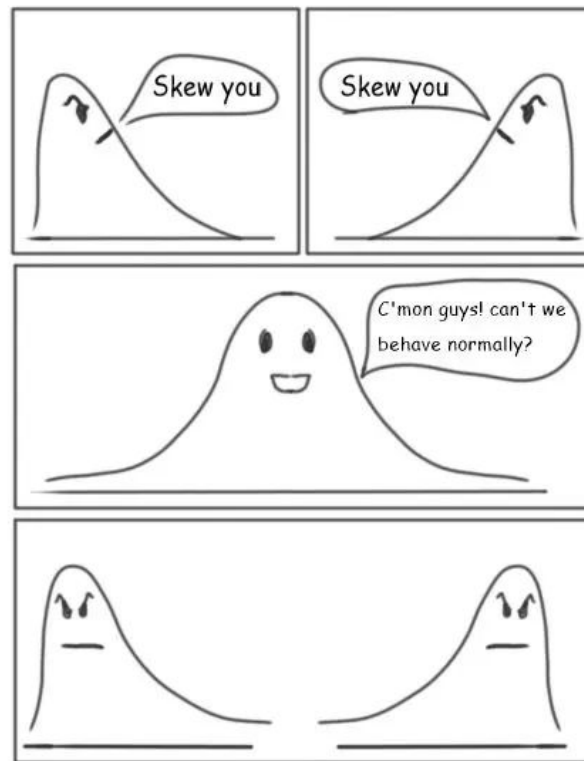
Our Workshop Python Package

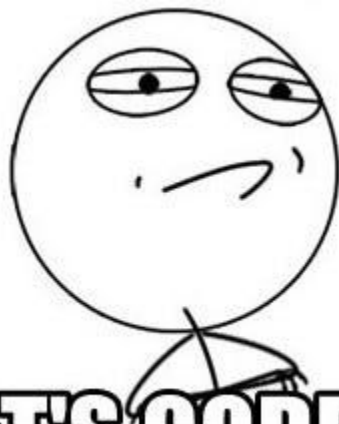




Our Workshop Python Package

- Reads in some data
- Calculates **metrics** based on the data (min, max...)





LET'S CODE IT



Pick your Poison



Visual Studio Code



Cursor IDE

An AI-powered code editor



Create repo on GitHub

1. Go to Github: <https://github.com/>
2. Create a new repo: use the suggested random name and add "data-analytics" in front, e.g.
squiggly-groundhog-data-analytics
3. Public (needed for GitHub Pages to work)
4. .gitignore for Python
5. Add a license (e.g. MIT)
6. Add a README.md
7. Clone repo locally in Visual Studio Code/Cursor
8. Press F1, enter "Install Extensions" and install the Python extension



Create package structure

1. Make sure the folder `.vscode` is in the `.gitignore` file
2. Install Python 3.13
`uv python install 3.13`
3. Create package structure:
`uv init --name data-analytics-<random name> --package`
4. Create virtual environment:
`uv venv`
5. Select Python interpreter in your IDE

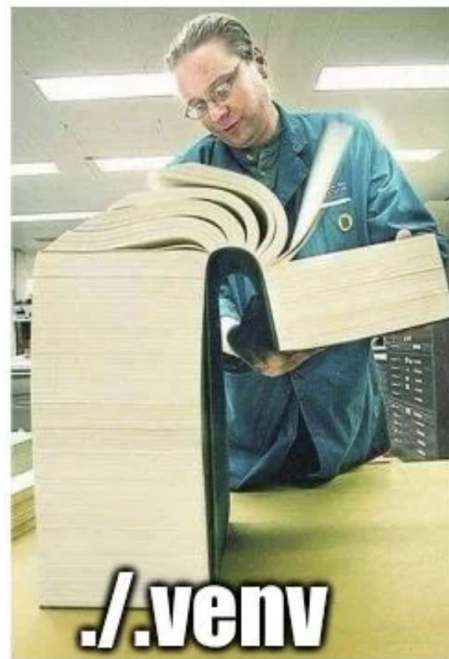


The .venv Folder

A **virtual environment** for your project

- Separate from your global Python
- Keeps dependencies isolated per project
- What's inside?
 - **bin/** or **scripts/** → executables
 - **lib/** → installed packages
 - Python runtime

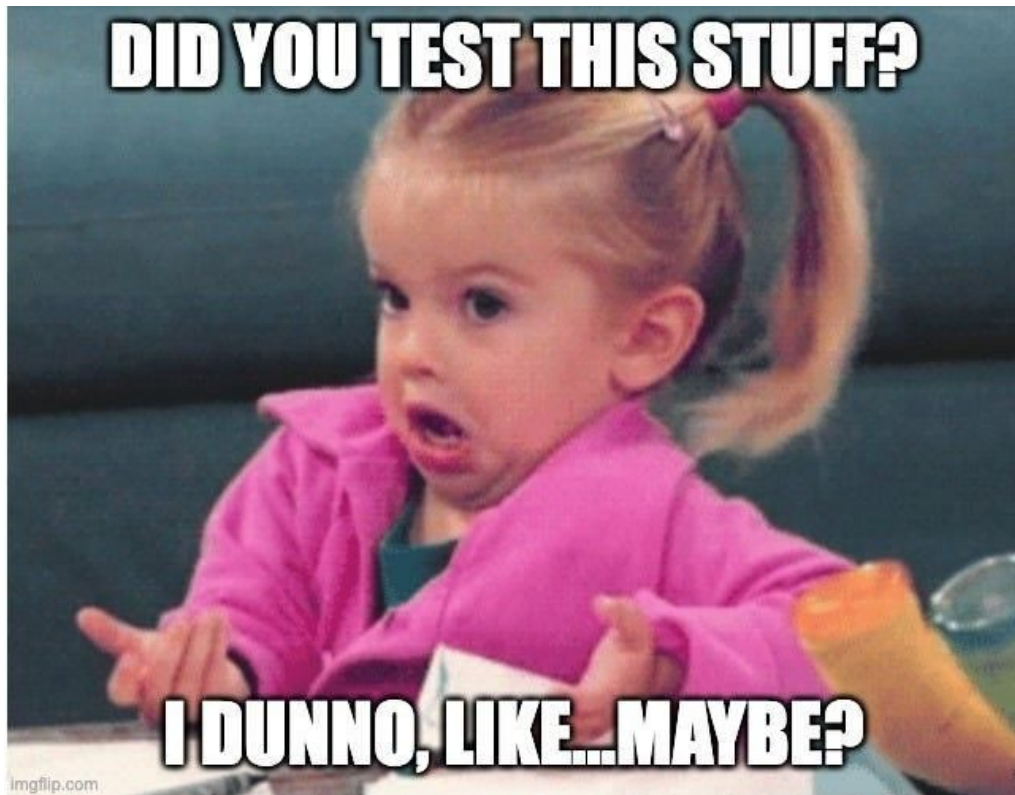
PYTHON PROJECTS BE LIKE



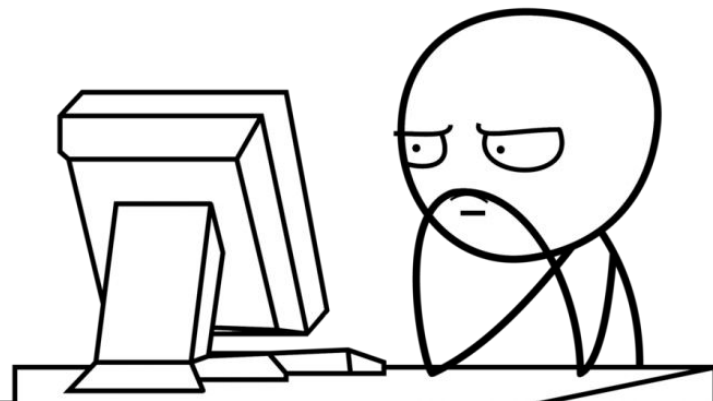


Set up Dependencies

1. Add pandas as dependency:
`uv add pandas`
2. Check dependencies:
`uv tree`



Seems I should
write some
tests...





03

Testing & Pre-Commit Hooks

`pytest vs. unittest and pre-commit`



Test Package

unittest

- in **Python standard library**
- **class-based** structure

pytest

- simple syntax
- tests are **plain functions**
- powerful features:
fixtures, parametrisation

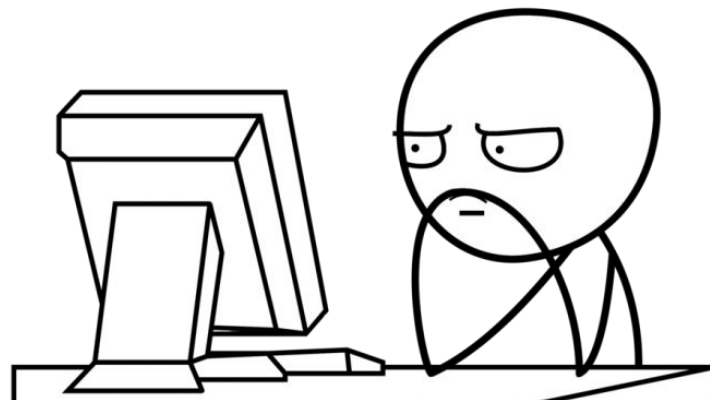


Set up Pytest

1. Install pytest package:
`uv add pytest --group dev`
2. Create folder tests in the root directory of the package
3. Implement tests
4. Set up the tests in VSCode/Cursor (choose pytest and the tests folder)
5. Alternatively, call:
`uv run pytest`



I want my code
to be
beautiful...





Pre-commit Hooks

What is a pre-commit hook?

- Runs checks **before every git commit**
- Helps catch issues early



Why?

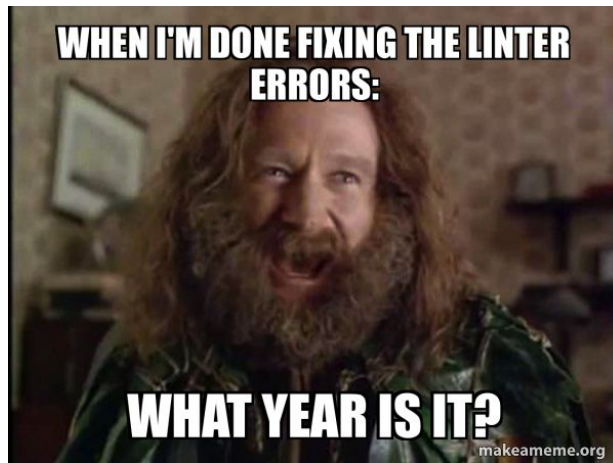
- Consistent code style
- Fewer bugs
- No **"forgot to format"** moments
- Same checks for everyone





Pre-commit in Python

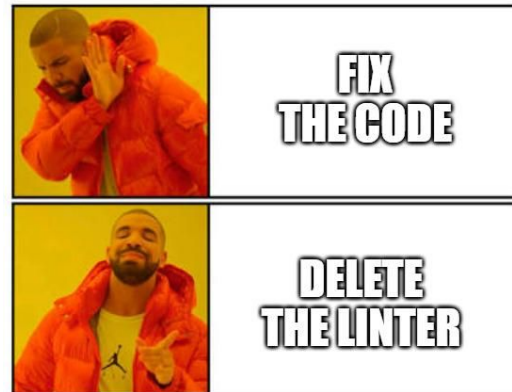
- Install `package pre-commit`
- Configure hooks using `.pre-commit-config.yaml`
 - flake8 → linting
 - black → code formatting
 - isort → import sorting
 - pytest → run tests
 - mypy → type checking
- Try to commit!





Set up Pre-commit

1. Install pre-commit package:
`uv add pre-commit --group dev`
2. Go to GitHub: <https://github.com/kolbl/data-analytics>
3. Copy the file `.pre-commit-config.yaml`
4. Install the configured pre-commit hooks:
`uv run pre-commit install`
5. Try to commit:
`git add .`
`git commit -m "adding package code"`



imgflip.com



04

Build & Publish

Build and publish the package to a Python package registry

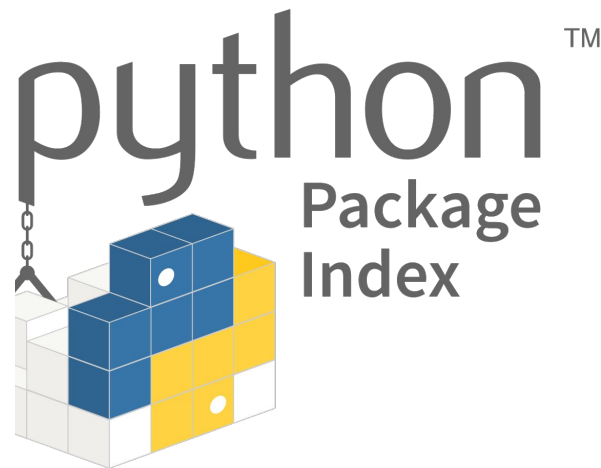




PyPI

Python Package Index - a central place to store and share Python packages

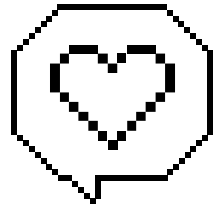
- **Public** Python package registry
- Publish your own package
- Others can install the package easily with **pip, poetry or uv**





> 8.400.000

Python packages on PyPI: <https://pypi.org/>

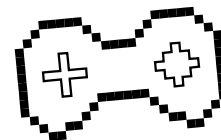




Private Python Package Registry

A package registry that is **not public**

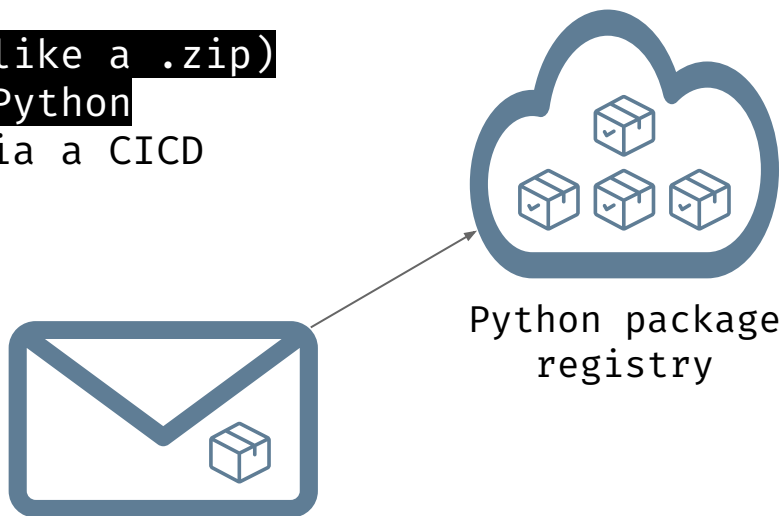
- Used **inside companies or teams**
- Store **internal or proprietary packages**
- Common options
 - Artifactory
 - Nexus
 - GitHub Packages
 - Self-managed Gitlab





Build & Publish

1. Package needs to be build (like a .zip)
2. Package gets uploaded to a Python package registry (usually via a CICD pipeline)





Build & Publish



test

```
uv run pytest
```



build

```
uv build
```



publish

```
uv publish
```



download

```
uv add data-analytics
```





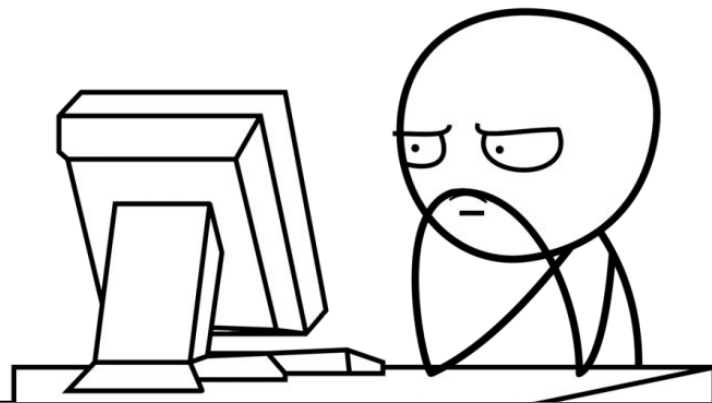
Create Account on TestPyPI

1. Go to <https://test.pypi.org/>
2. Create an account
3. Create a token
4. Add it to GitHub Secrets: Settings > Secrets and variables > Actions > section “Repository Secrets”
5. New repository secret called: PYPI_API_TOKEN

Docs: <https://packaging.python.org/en/latest/guides/using-testpypi/>



People are
asking how to
use my
package...





Set up MkDocs

1. Go to GitHub: <https://github.com/kolbl/data-analytics>
2. Copy the file `mkdocs.yaml` and the folder “docs”
3. Adapt files in docs folder to your project
4. Edit `mkdocs.yaml`
`edit_uri: <your GitHub repo URL>/edit/main/docs/`



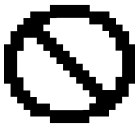
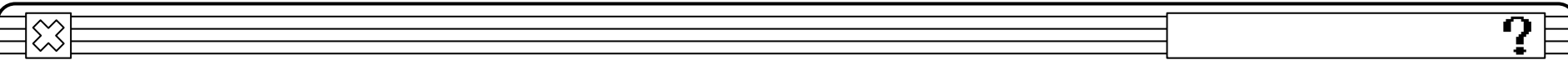
CICD Pipeline

1. Go to <https://github.com/kolbl/data-analytics>
2. Copy `.github` folder to your root directory
3. Adjust any paths in `ci.yaml` file
4. Enable GitHub Pages: Settings > Pages > Build and deployment
> select “GitHub Actions” as source



Whoa!

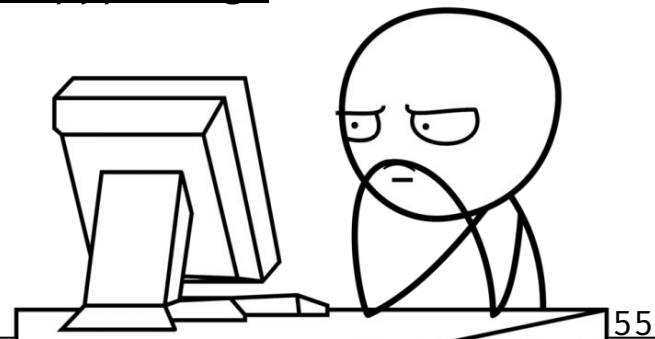
We did it - we published our first Python
package to a **Python package registry!**
Search for the package: <https://test.pypi.org/>



Whoa!

We did it - we published our first Python
package to a **Python package registry**!
Search for the package: <https://test.pypi.org/>

Now - how
can I use
this magic?





05

Package Integration

Install and use the package in another Python project



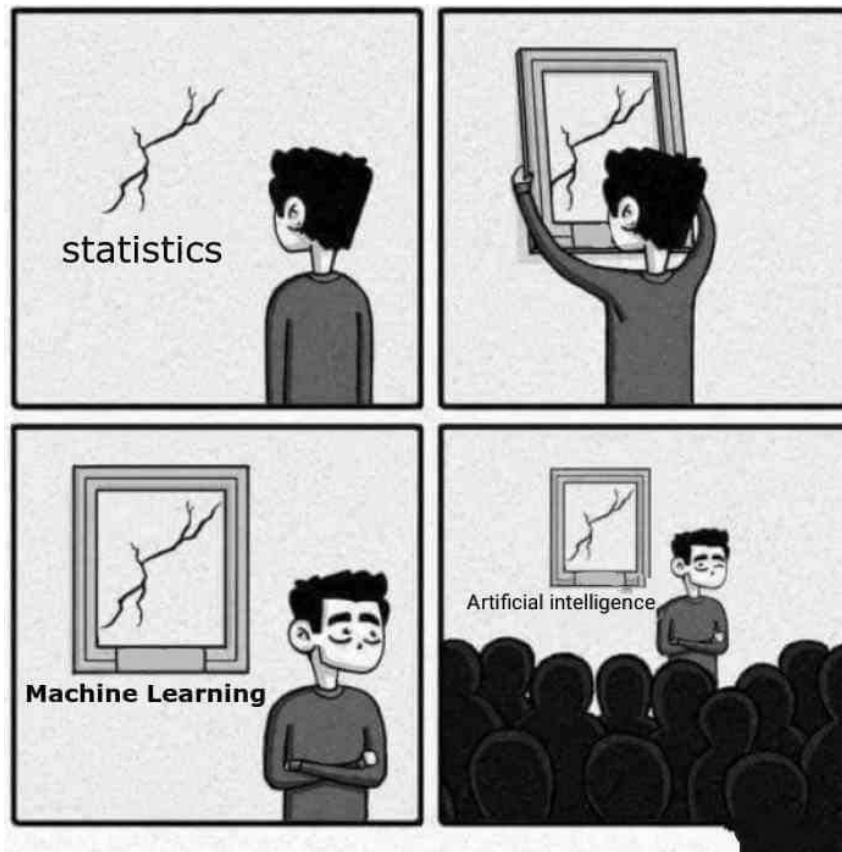
Using your package in another project

1. Install the package:

```
uv add my-package
```

2. Import it in your code:

```
from my_package import my_function  
result = my_function()  
print(result)
```





Our workshop project

Groundhog Day

Traditionally, on **February 2nd**, various groundhogs across North America are consulted by local public figures for an **annual weather prediction**:

- If the groundhog **'sees its shadow'**, it means 6 more weeks of winter
- If the groundhog **doesn't see its shadow**, spring will come early

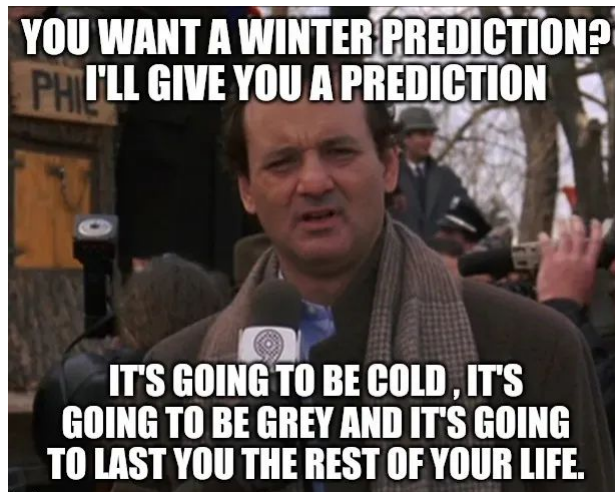




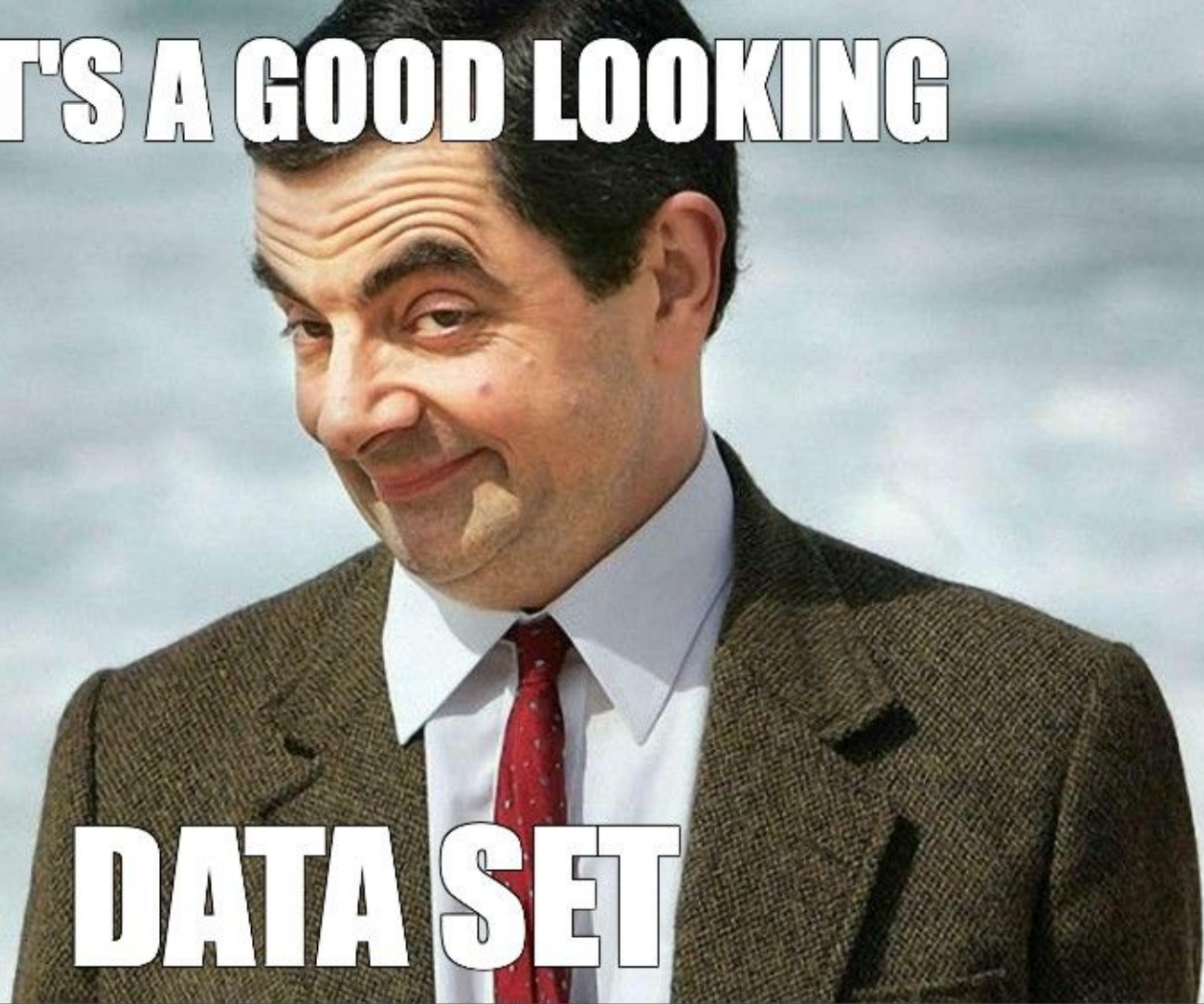
Our workshop project

Groundhog Forecast Dashboard

- A small dashboard built with **Streamlit**
- Uses a public API based on predictions from **Groundhog Day**:
<https://groundhog-day.com/>
- Displays infos about the groundhogs and their prediction



THAT'S A GOOD LOOKING



DATA SET



Our workshop project

Streamlit Dashboard

- Python → Web app
- Fast prototyping

Integrate Python package

Install our analytics package to get some **statistics** from the Groundhog Day API



Package Integration

1. Clone repo from <https://github.com/kolbl/streamlit-app>
2. Create virtual environment and install dependencies from `pyproject.toml`:
`uv sync`
3. Remove existing package:
`uv remove data-analytics`
4. Select Python Interpreter and install the package:
`uv add "data-analytics<... >=0.1.0.dev1" --extra-index-url https://test.pypi.org/simple/ --index-strategy unsafe-best-match`
5. Import it in `src/app.py`:
`from data_analytics_<...> import calculate_min_of_column`



Run Streamlit

1. Run streamlit app:
`uv run streamlit run src/app.py`
2. In browser open: <http://localhost:8501/>



Like a machine learning
model but **furrier**





AM I TESTING THE CODE



OR IS THE CODE TESTING ME





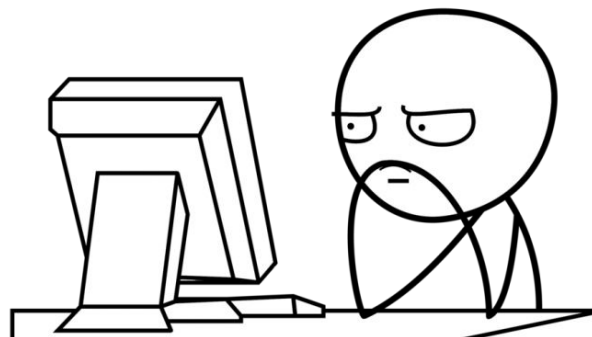
AM I TESTING THE CODE



OR IS THE CODE TESTING ME



Colleague says the
package code does
not work on their
machine...





06

Dev Containers

Set up a Dev Container for your Python package development



Dev Containers

A **preconfigured development environment** in a container

- Project includes a **.devcontainer** configuration
- Container is built based on Dockerfile
- You **develop inside** the container
- Everyone gets the same environment
- Works across OS (Linux, macOS, Windows)

Like **Docker**... but for your development setup!





Setup your Dev Container

1. Install Docker (alternative: Podman/Podman Desktop):
Docker Desktop for Windows:
<https://docs.docker.com/desktop/setup/install/windows-install/>
Docker Desktop for Mac:
<https://docs.docker.com/desktop/setup/install/mac-install/>
Docker Engine for Linux:
<https://docs.docker.com/engine/install/>
2. Go to <https://github.com/kolbl/data-analytics>
3. Download the folder called `.devcontainer`
4. Copy it into your Python package repo
5. In VSCode: install Dev Containers extension
6. Press F1 and enter "Dev Containers: Reuild and Reopen in Container"



Alternative: GitHub Codespace

1. Commit and push the `.devcontainer` folder
2. Go to your GitHub repo
3. Create a GitHub codespace there (you can delete it later)



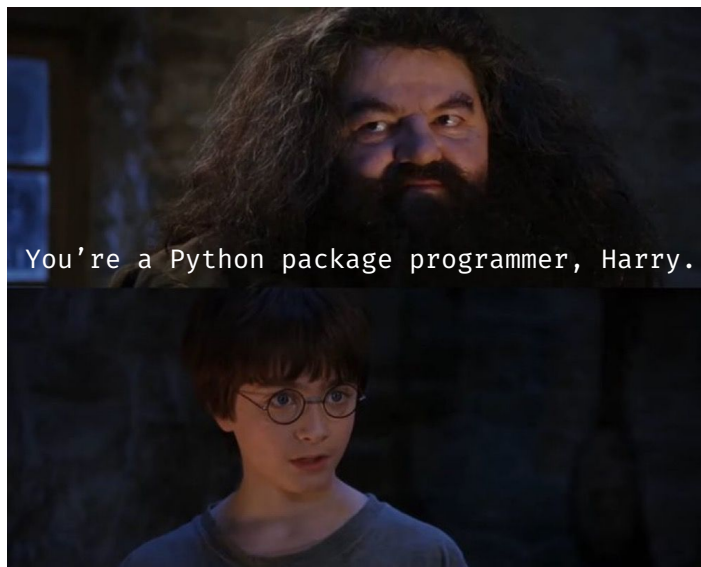
Add “span”

1. Now it's your turn!
2. We implemented min and max together - now implement **“span”** (max - min) on your own!
3. Build the package via CICD
4. Update the package and include the new code in Streamlit!





Whoa!





What we did today

created package
structure

_____.

added
dependencies



wrote package
code

tested the code

_____.



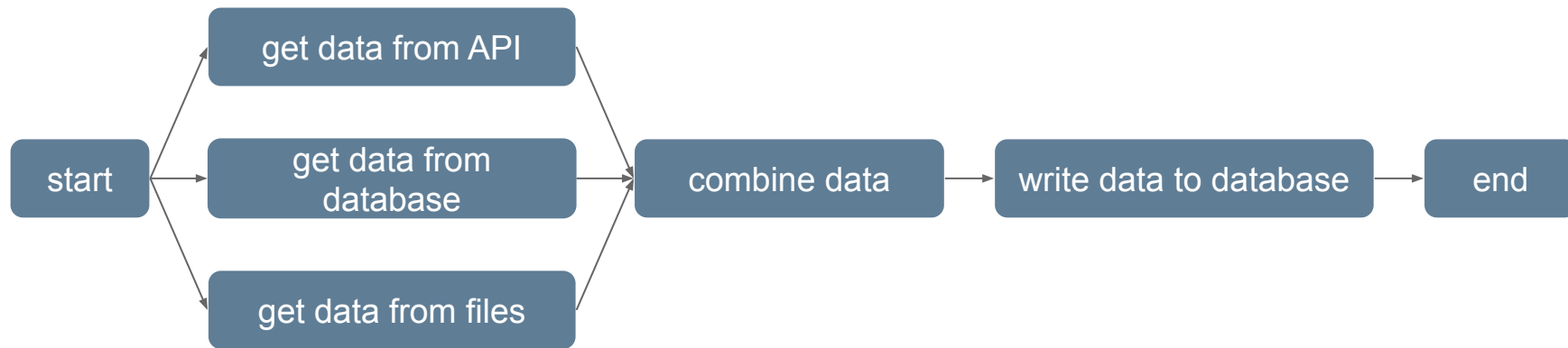
built and
published the
package

_____.

integrated the
package

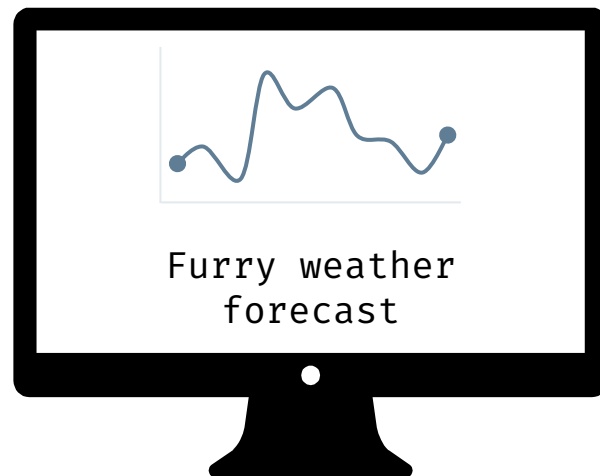


ETL Pipeline





ETL Pipeline





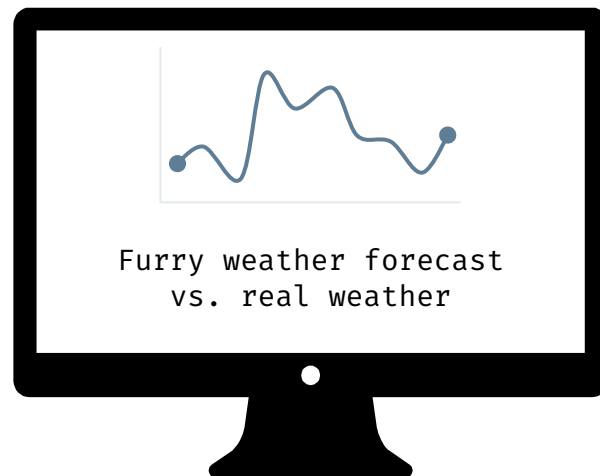
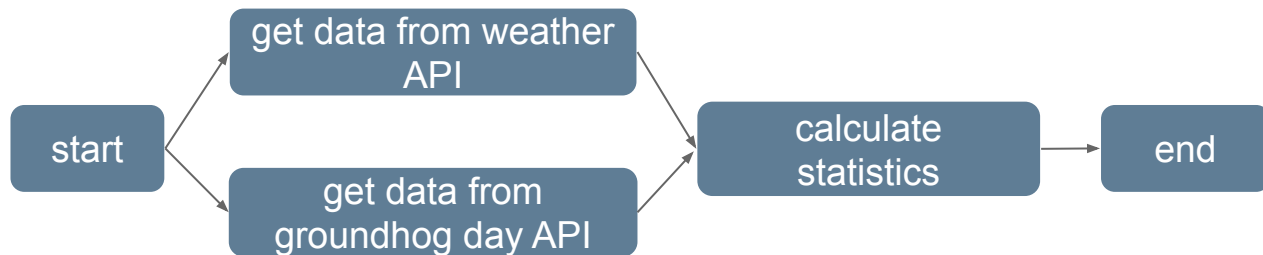
Lies, damned lies and **statistics**



Mark Twain



ETL Pipeline





A wide-angle shot of a modern classroom or workshop. In the foreground, the backs of several students' heads are visible as they sit at desks. They are looking towards a large projector screen at the front of the room. The screen displays a code editor with JavaScript code. The code includes comments in Chinese and defines a function 'createSnapshot' that interacts with a database and a file system. A projector is mounted on the ceiling, casting the screen's content. To the left, there are large windows with white horizontal blinds, letting in natural light. A small whiteboard is visible on the right side of the room. The overall atmosphere is one of a hands-on learning environment.

Hands-on workshops

A woman with dark hair tied back, wearing glasses and a black V-neck shirt, is speaking at a conference. She has a small microphone clipped to her shirt and is gesturing with her hands. The background is a blue screen with white text.

Awesome speakers



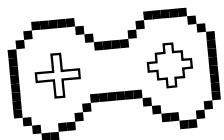
devops days GRAZ

4. - 5. September
2026

FH Campus02 &
WIFI, Graz

<https://www.devopsdays.at/>





Thanks!

Do you have any questions?

LinkedIn:

